Week 4 · Class 1

Transforming Variables & Causality

Sean Westwood

Segment A — Transforming and Creating Variables

In Today's Class

Segment A — Transforming and Creating Variables

- Creating new variables with `mutate()`
- Understanding conditional logic and comparison operators
- Variable recoding with `if_else()` and `case_when()`
- Calculating proportions with `summarise()`
- Extracting values with `pull()`

Segment B — Causality

- The three requirements for establishing causality
- Distinguishing between association and causation
- Temporal ordering in causal claims
- Identifying confounding variables and spurious correlations

Why Must We Transform Data?

Real Political Science Data is Messy

Example: Survey responses about income:

- Raw: `$35,000`, `$45000`, `45k`, `forty-five thousand`
- Analysis needs: `35000`, `45000`, `45000`, `45000`

Example: Education levels to numeric:

- Raw: `high school`, `HS grad`, `12 years`, `completed secondary`
- Analysis needs: All coded as `12` (years of education)

Example: Age data:

- Raw: Birth years like `1985`, `1992`, `1967`
- Analysis needs: Age groups like `"18-29"`, `"30-44"`, `"45-64"`

The transformation process

1. **Clean** inconsistent formats
2. **Recode** into meaningful categories
3. **Calculate** new measures (ratios, percentages)

The majority of data analysis time is spent on data transformation!

Creating New Variables

Creating Variables with mutate()

The `mutate()` function adds new columns to your dataframe

Basic syntax:

```
data %>%
  mutate(
    new_variable = calculation,
    another_var = some_function(existing_var)
  )
```

summarise() creates a new dataframe with new columns; mutate() alters the original dataframe

What mutate() does:

1. Takes your existing data
2. Adds new column(s) based on calculations
3. Keeps all original columns
4. Returns the expanded dataset

Mathematical Operations in mutate()

Basic math:

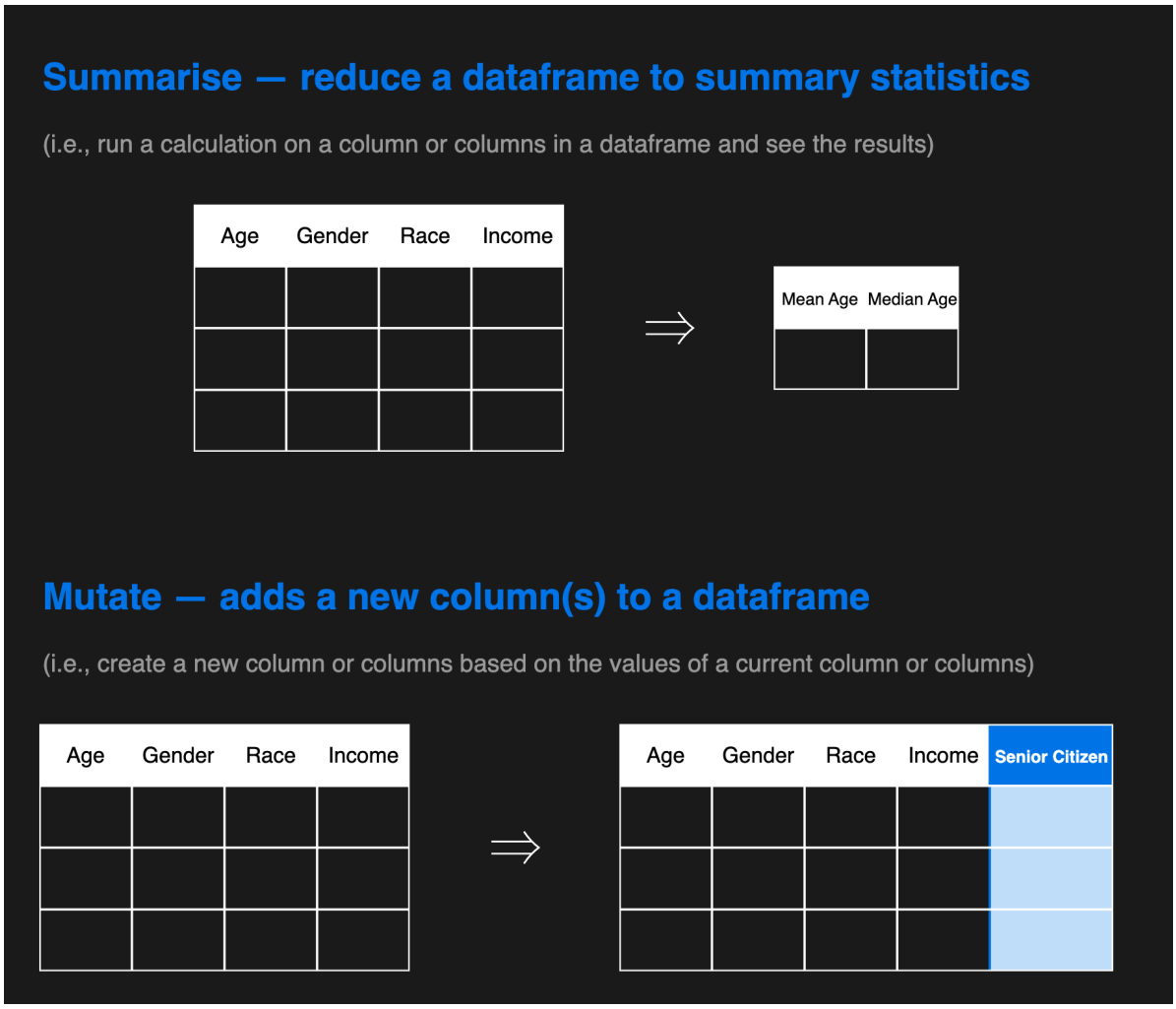
```
mutate(
  total = votes_dem + votes_rep,           # Addition
  margin = votes_dem - votes_rep,          # Subtraction
  vote_share = votes_dem / total_votes,    # Division
  doubled = approval_rating * 2            # Multiplication
)
```

This is row-wise! Each calculation will be independent for each row.

Useful calculations for political science:

```
mutate(
  turnout_rate = votes_cast / eligible_voters,
  spending_per_vote = total_spent / votes_received,
  ideology_squared = ideology_score^2,
  log_income = log(household_income)
)
```

Recap: Summarise v. Mutate



Understanding Conditional Logic

What is Conditional Logic?

In order to create new variables that are useful for analysis, we often need to use conditional logic.

Conditional logic: Making decisions based on conditions

In everyday life:

- *If* it's raining, *then* bring an umbrella
- *If* you're 21 or older, *then* you can drink alcohol
- *If* it is after 6:00 AM and before 10:00AM, *then* you can order breakfast at McDonalds

In R:

- *If* `party == "Democrat"`, *then* assign `"Liberal"`
- *If* `age >= 65`, *then* assign `"Senior"`
- *If* `approval_rating > 50`, *then* assign `"Majority Approves"`

The Building Blocks: Comparison Operators

Comparison operators:

- `==` (equal to)
- `!=` (not equal to)
- `>` (greater than)
- `>=` (greater than or equal to)
- `<` (less than)
- `<=` (less than or equal to)

The triad of 'equals'

- `<-` used to assign values to objects (make a variable 'get' a value)
- `=` Specify values when using a function (set an argumen to a value)
- `==` Test whether two values are equal (returning a logical result: TRUE, FALSE, or NA)

Why We Want TRUE/FALSE Outcomes

TRUE/FALSE is the foundation of data transformation because:

- All data decisions can ultimately be reduced down to "yes" or "no" questions
- We're essentially asking: "Does this observation meet our criteria?"
- These logical outcomes let us sort, categorize, and transform our data
- This binary logic is what allows us to create meaningful new variables from raw data with simple rules

Examples of TRUE/FALSE Outcomes

Equal to: ==

```
party_id == "Republican"
# TRUE if Republican, FALSE otherwise
```

Not equal to: !=

```
education != "College"
# TRUE if not College, FALSE if College
```

Greater than: >

```
age > 65
# TRUE if older than 65
```

Greater than or equal: >=

```
income >= 50000
# TRUE if 50,000 or more
```

Less than: <

```
approval < 40
# TRUE if less than 40
```

Less than or equal: <=

```
years_in_office <= 2
# TRUE if 2 years or fewer
```

Understanding TRUE and FALSE

Every comparison returns TRUE or FALSE:

```r
# Examples with actual values
age <- 25
age >= 18        # Returns: TRUE
age >= 65        # Returns: FALSE

party <- "Democrat"
party == "Republican"    # Returns: FALSE
party != "Republican"    # Returns: TRUE
```

R treats TRUE as 1 and FALSE as 0:

We can apply mathimatical operators with TRUE/FALSE:

- `sum(TRUE, FALSE, TRUE)` = 2 (counts the TRUEs)
- `mean(TRUE, FALSE, TRUE)` = 0.67 (proportion that are TRUE)

Why mean() gives proportions: Since TRUE = 1 and FALSE = 0, the mean is the sum divided by the total count, which is exactly what a proportion is!

Detailed Operator Examples

Example 1: Creating a New Variable with == (Equal To)

Question: How many survey respondents identify as Democrats?

```r
# Sample data
survey_sample <- tibble(
  respondent = 1:8,
  party_id = c("Democrat", "Republican", "Democrat", "Independent",
               "Democrat", "Republican", "Democrat", "Independent")
)

# Show the data
survey_sample
```

```
# A tibble: 8 x 2
  respondent party_id
       <int> <chr>
1          1 Democrat
```

```
2           2 Republican
3           3 Democrat
4           4 Independent
5           5 Democrat
6           6 Republican
7           7 Democrat
8           8 Independent
```

Using == to Find Democrats

```
# Test each row: is party_id equal to "Democrat"?
survey_sample %>%
  mutate(
    is_democrat = party_id == "Democrat"
  )
```

```
# A tibble: 8 x 3
  respondent party_id    is_democrat
       <int> <chr>       <lgl>
1          1 Democrat    TRUE
2          2 Republican  FALSE
3          3 Democrat    TRUE
4          4 Independent FALSE
5          5 Democrat    TRUE
6          6 Republican  FALSE
7          7 Democrat    TRUE
8          8 Independent FALSE
```

What happened:

- Row 1: `"Democrat" == "Democrat"` → `TRUE`
- Row 2: `"Republican" == "Democrat"` → `FALSE`

- Row 3: `"Democrat" == "Democrat"` → `TRUE`
- And so on...

Counting with ==

```r
survey_sample %>%
  summarise(
    total_respondents = n(),
    democrats = sum(party_id == "Democrat"),
    democrat_percentage = mean(party_id == "Democrat") * 100
  )
```

```
# A tibble: 1 x 3
  total_respondents democrats democrat_percentage
              <int>     <int>                <dbl>
1                 8         4                   50
```

- `sum(party_id == "Democrat")` counts TRUE values
- `mean(party_id == "Democrat") * 100` gives the proportion of Democrats

Example 2: The != (Not Equal To) Operator

Question: How many respondents are NOT Republicans?

```r
# Count non-Republicans
survey_sample %>%
  summarise(
    non_republicans = sum(party_id != "Republican"),
    non_republican_pct = mean(party_id != "Republican") * 100
  )
```

```
# A tibble: 1 x 2
  non_republicans non_republican_pct
            <int>              <dbl>
1               6                 75
```

Example 3: Using >= to Find Seniors

```
# A tibble: 10 x 2
   voter_id   age
      <int> <dbl>
 1        1    22
 2        2    35
 3        3    67
```

```
 4         4    45
 5         5    72
 6         6    29
 7         7    81
 8         8    55
 9         9    19
10        10    65
```

```r
# Count seniors
age_sample %>%
  summarise(
    total_voters = n(),
    seniors = sum(age >= 65),
    senior_percentage = mean(age >= 65) * 100
  )
```

```
# A tibble: 1 x 3
  total_voters seniors senior_percentage
         <int>   <int>             <dbl>
1           10       4                40
```

Example 4: The <= (Less Than or Equal) Operator

Question: How many voters are young adults (30 or younger)?

```r
# Count young adults
age_sample %>%
  summarise(
    young_adults = sum(age <= 30),
    young_adult_pct = mean(age <= 30) * 100
  )
```

```
# A tibble: 1 x 2
  young_adults young_adult_pct
         <int>           <dbl>
1            3              30
```

Multiple Conditions

The world is complex - we often need to examine multiple variables simultaneously

Real-world questions require multiple conditions:

- Young AND Democrat voters (age < 30 AND party == "Democrat")
- High achievers (high IQ AND admitted to Dartmouth)
- At-risk populations (low income AND poor health)
- Swing voters (independent AND frequent voters)

Combining Conditions with AND (&) and OR (|)

AND (`&`): Both conditions must be TRUE

```
age >= 18 & age < 65    # Working age adults
```

OR (`|`): Either condition can be TRUE

```
party == "Democrat" | party == "Republican"    # Major party members
```

Example of multiple conditions with real data

```
# Complex conditions
age_sample %>%
  mutate(
    working_age = age >= 18 & age < 65,
    voting_age = age >= 18
  ) %>%
  summarise(
    working_age_count = sum(working_age),
    voting_age_count = sum(voting_age)
  )
```

```
# A tibble: 1 x 2
  working_age_count voting_age_count
              <int>            <int>
1                 6               10
```

Putting This All Together

Logical Operations in mutate()

Creating TRUE/FALSE variables:

```
mutate(
  is_competitive = vote_margin < 0.05,     # TRUE if close race
  high_turnout = turnout > 0.7,            # TRUE if high turnout
  experienced = years_in_office >= 4,      # TRUE if experienced
  young_high_turnout = age < 30 & turnout > 0.7,        # Young AND high turnout
  major_party_experienced = (party == "Democrat" | party == "Republican") & years_in_office >
)
```

Why create logical variables?:

- Easy to count: `sum(is_competitive)`
- Easy to get percentages: `mean(high_turnout)`
- Clear for analysis: `filter(major_party)`

The if_else() Function

What if we want to create a new variable that is a categorical variable (e.g., "Senior", "Non-Senior")?

Instead of creating a variable that shows is someone is young or not, we can create a varaible that calls them "Young" or "Old".

For simple binary decisions:

```
mutate(
  age_group = if_else(age >= 65, "Senior", "Non-Senior"),
  result = if_else(vote_share > 0.5, "Won", "Lost"),
  income_level = if_else(income >= 50000, "High", "Low")
)
```

Syntax: `if_else(condition, value_if_true, value_if_false)`

When to use if_else():

- Only two possible outcomes
- Simple, clear conditions where the labels are sensible

The case_when() Function

Real political data often has **multiple meaningful categories** that can't be captured with simple TRUE/FALSE:

Example 1: Education levels:

- Raw data: Years of education (8, 12, 14, 16, 18, 20)
- Need categories: "Less than HS", "High School", "Some College", "Bachelor's", "Graduate"

Example 2: Congressional ideology scores:

- Raw data: DW-NOMINATE scores (-0.8, -0.2, 0.1, 0.6, 0.9)
- Need categories: "Very Liberal", "Liberal", "Moderate", "Conservative", "Very Conservative"

case_when() handles multiple conditions in order:

Using case_when()

For multiple conditions and outcomes:

What if we wanted to make a new variable called "age_category" by checking each person's age and assigning them to one of four age groups based on which condition they meet first?

```
mutate(
  age_category = case_when(
    age < 30 ~ "Young",
    age < 50 ~ "Middle-aged",
    age < 70 ~ "Older",
    TRUE ~ "Senior"  # catch-all for age >= 70
  )
)
```

Key rules for case_when():

1. Conditions are tested **in order**
2. First TRUE condition wins
3. Use ~ to separate condition from result
4. Always end with `TRUE ~ "catch-all"` for safety

Understanding the ~ Symbol

The tilde (~) means "then":

- `age < 30 ~ "Young"` means "if age < 30, then assign 'Young' "
- `income >= 100000 ~ "High"` means "if income $>= 100000$, then assign 'High' "

Think of it as an arrow:

- `condition ~ result`
- "When this condition is TRUE, give this result"

Only used in case_when(), not in if_else()

case_when() vs if_else()

Use if_else() for 2 categories:

```
party_type = if_else(party == "Independent", "Independent", "Major Party")
```

Use case_when() for 3+ categories:

```
party_group = case_when(
  party == "Democrat" ~ "Democrat",
  party == "Republican" ~ "Republican",
  party == "Independent" ~ "Independent",
  TRUE ~ "Other"
)
```

Both create new columns, but case_when() is more flexible

Common case_when() Patterns

Age groups:

```
age_group = case_when(
  age < 25 ~ "18-24",
  age < 35 ~ "25-34",
  age < 50 ~ "35-49",
  age < 65 ~ "50-64",
  TRUE ~ "65+"
)
```

Income brackets:

```
income_bracket = case_when(
  income < 25000 ~ "Low",
  income < 75000 ~ "Middle",
  TRUE ~ "High"
)
```

Performance ratings:

```
rating = case_when(
  score >= 90 ~ "Excellent",
  score >= 80 ~ "Good",
  score >= 70 ~ "Fair",
  TRUE ~ "Poor"
)
```

Examples: Summarize Without Mutate

Example 1: Congressional Voting Patterns Analysis

Research Question: What are the basic characteristics of congressional voting behavior by party?

Data: DW-NOMINATE dataset with 1,000 members of Congress, including ideology scores, party affiliation, chamber, and state information.

Code:

```
congress_data %>%
  group_by(party_code) %>%
  summarise(
    count = n(),
    mean_ideology = mean(nominate_dim1, na.rm = TRUE),
    median_ideology = median(nominate_dim1, na.rm = TRUE),
    percentage = n() / nrow(congress_data) * 100
  )
```

Results:

```
# A tibble: 3 x 5
  party_code  count mean_ideology median_ideology percentage
  <chr>       <int>         <dbl>           <dbl>      <dbl>
1 Democrat      440        -0.363          -0.366       44
2 Independent    42        -0.251          -0.254        4.2
3 Republican    518         0.420           0.408       51.8
```

AI Prompts for This Congressional Analysis

Prompt to Create Code:

```
"I have congressional data that look like this:

Rows: 1000Rows: 1,000
Columns: 5
$ member_id     <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 1…
$ party_code    <chr> "Democrat", "Republican", "Democrat", "Democrat", "Repub…
...

Create tidyverse code to summarize by party: count of members, mean and median ideology scor
```

AI Response:

```
I'll help you create a summary by party using group_by() and summarise(). Here's the code:

[code provided above]
```

Prompt to Explain Code:

```
"Explain this summarise code step by step. What does each function do and why do we use na.r
```

AI Response:

```
Let me break this down:

1. group_by(party_code): Creates separate
   groups for each party
2. n(): Counts rows in each group
3. mean(nominate_dim1, na.rm=TRUE): Calculates
   average ideology, ignoring missing values
```

```
4. na.rm=TRUE prevents errors if any ideology
   scores are missing (NA)
5. percentage calculation: n() divided by
   total dataset size * 100
```

Extracting Values with pull()

When to Use pull()

Purpose: Extract a single column as a vector

Common use cases:

- Get a single statistic for further calculation
- Extract values for comparison
- Create variables from summary statistics

Basic syntax:

```
data %>%
  pull(column_name)

# Or extract calculated values
data %>%
  summarise(mean_value = mean(variable)) %>%
  pull(mean_value)
```

Example: Using pull() for Calculations

```
# Extract median income for threshold calculation
median_income <- survey_data %>%
  summarise(median_inc = median(income)) %>%
  pull(median_inc)

median_income
```

```
[1] 51198.99
```

```r
# Use extracted value in new calculation
survey_data %>%
  mutate(
    above_median = income > median_income,
    income_ratio = income / median_income
  ) %>%
  select(respondent_id, income, above_median, income_ratio) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 x 4
  respondent_id  income above_median income_ratio
          <int>   <dbl> <lgl>                <dbl>
1             1  35899. FALSE                0.701
2             2 136369. TRUE                 2.66
3             3  43636. FALSE                0.852
4             4  27250. FALSE                0.532
5             5  46590. FALSE                0.910
```

Advanced Conditional Logic

Example: Complex Campaign Classifications

Research Question: How can we classify campaigns by competitiveness and resource level?

```r
# Create sample campaign data with more variables
campaign_complex <- tibble(
  candidate = paste("Candidate", 1:10),
  vote_share = c(0.52, 0.48, 0.67, 0.33, 0.51, 0.49, 0.78, 0.22, 0.55, 0.45),
  total_spent = c(150000, 145000, 300000, 100000, 180000, 175000, 500000, 80000, 200000, 190
  incumbent = c(TRUE, FALSE, TRUE, FALSE, FALSE, TRUE, TRUE, FALSE, FALSE, TRUE)
)

campaign_complex
```

```
# A tibble: 10 x 4
   candidate    vote_share total_spent incumbent
   <chr>             <dbl>       <dbl> <lgl>
 1 Candidate 1        0.52      150000 TRUE
```

```
 2 Candidate 2        0.48      145000 FALSE
 3 Candidate 3        0.67      300000 TRUE
 4 Candidate 4        0.33      100000 FALSE
 5 Candidate 5        0.51      180000 FALSE
 6 Candidate 6        0.49      175000 TRUE
 7 Candidate 7        0.78      500000 TRUE
 8 Candidate 8        0.22       80000 FALSE
 9 Candidate 9        0.55      200000 FALSE
10 Candidate 10       0.45      190000 TRUE
```

Creating Complex Classifications with case_when()

```
campaign_complex %>%
  mutate(
    # Competitiveness based on vote margin
    competitiveness = case_when(
      abs(vote_share - 0.5) <= 0.02 ~ "Extremely Close",  # Within 2%
      abs(vote_share - 0.5) <= 0.05 ~ "Competitive",      # Within 5%
      abs(vote_share - 0.5) <= 0.10 ~ "Somewhat Safe",    # Within 10%
      TRUE ~ "Safe"                                        # More than 10%
    ),

    # Spending level
    spending_level = case_when(
      total_spent >= 400000 ~ "High Budget",
      total_spent >= 200000 ~ "Medium Budget",
      total_spent >= 100000 ~ "Low Budget",
      TRUE ~ "Minimal Budget"
    ),

    # Advantage type using multiple conditions
    advantage = case_when(
      incumbent == TRUE & total_spent >= 200000 ~ "Incumbent + Money",
      incumbent == TRUE & total_spent < 200000 ~ "Incumbent Only",
      incumbent == FALSE & total_spent >= 200000 ~ "Money Only",
      TRUE ~ "Neither Advantage"
    )
  ) %>%
  select(candidate, vote_share, competitiveness, spending_level, advantage)
```

```
# A tibble: 10 x 5
```

```
   candidate    vote_share competitiveness spending_level advantage
   <chr>             <dbl> <chr>           <chr>          <chr>
 1 Candidate 1        0.52 Competitive     Low Budget     Incumbent Only
 2 Candidate 2        0.48 Competitive     Low Budget     Neither Advantage
 3 Candidate 3        0.67 Safe            Medium Budget  Incumbent + Money
 4 Candidate 4        0.33 Safe            Low Budget     Neither Advantage
 5 Candidate 5        0.51 Extremely Close Low Budget     Neither Advantage
 6 Candidate 6        0.49 Extremely Close Low Budget     Incumbent Only
 7 Candidate 7        0.78 Safe            High Budget    Incumbent + Money
 8 Candidate 8        0.22 Safe            Minimal Budget Neither Advantage
 9 Candidate 9        0.55 Somewhat Safe   Medium Budget  Money Only
10 Candidate 10       0.45 Competitive     Low Budget     Incumbent Only
```

Understanding Complex Conditions

Breaking down the `advantage` classification:

1. `incumbent == TRUE & total_spent >= 200000`
    - Must be incumbent AND high spending
    - Both conditions must be TRUE
2. `incumbent == TRUE & total_spent < 200000`
    - Must be incumbent AND low spending
    - Checked only if #1 is FALSE
3. `incumbent == FALSE & total_spent >= 200000`
    - Must be challenger AND high spending
    - Checked only if #1 and #2 are FALSE
4. `TRUE`
    - Catches all remaining cases
    - Challenger with low spending

Where to Get More Reps

You now have every tool this segment teaches: `mutate()`, the comparison operators, `if_else()`, `case_when()`, and `pull()`.

Don't memorize patterns — recognize the shape:

- Need a *new variable*? → `mutate()`
- *Two* outcomes? → `if_else()`

- *Three or more* outcomes? → `case_when()`

Tonight's exercise opens with a fully worked recoding example you can use as a template, then has you build your own transformations — with AI assistance for the harder ones.

This week's reading walks through each pattern in depth, with code you can copy and adapt.

Best Practices

Common Mistakes to Avoid

Logical operator confusion:

- Use `&` for AND, `|` for OR in conditions
- Remember operator precedence with parentheses

case_when() ordering:

- Conditions are evaluated in order
- More specific conditions should come first
- Always include a catch-all with `TRUE`

Missing value handling:

- Use `na.rm = TRUE` in summary functions when needed
- Consider `is.na()` for missing value conditions

Segment B — Causality

What Does It Take to Show Causation?

Ice Cream Crime

In 2013, "researchers" noticed that counties with higher ice cream sales had higher rates of violent crime.

Should we ban ice cream to reduce crime?

Your intuition says: Probably not…

But why not? What's wrong with this reasoning?

This is a classic example of a **spurious correlation.**

- Hot weather increases both ice cream sales AND violent crime. Temperature is the **confounding variable** that explains both phenomena.
- Correlation Causation. Just because two things happen together doesn't mean one causes the other.

Correlation vs. Causation: What Can Each Tell Us?

Correlation isn't useless — it just answers a **different question** than causation.

Correlation lets us...

- **Predict**: see X, expect Y (high-margarine years tend to be high-divorce years)
- **Screen & flag**: spot at-risk cases worth a closer look
- **Describe** patterns in the world

No mechanism required — we don't need to know why.

Only causation lets us...

- **Predict the effect of acting**: if we *change* X, does Y move?
- **Justify policy**: "will this program actually work?"
- **Answer counterfactuals**: "what would have happened otherwise?"

Requires ruling out confounding and reverse causation.

Correlation answers **"what goes together?"** Causation answers **"what happens if we intervene?"** Mistaking one for the other is the costly error.

The Three Requirements for Causality

To claim that X causes Y, you need:

1. **Association**: X and Y are related (when X changes, Y changes)
2. **Temporal Ordering**: X happens before Y (cause before effect)
3. **No Confounding**: No other variable(s) Z causes both X and Y

All three are required. If any one is missing, you cannot make a causal claim.

Requirement 1: Association

Demonstrating Relationship

Association means: When X changes, Y systematically changes too

Examples of association:

- Higher campaign spending correlates with more votes
- More education correlates with higher income
- Social media use correlates with political polarization

Testing Associations with Data: Example 1 - Poverty and Savings

```
library(tidyverse)

# Load poverty and savings data
poverty <- read_csv("../../data/poverty.csv")
```

```
Rows: 2670 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (1): treatment
dbl (4): cash, accts_amt, stroop_time, income_less20k

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Test association: Does savings vary by poverty status?
poverty_savings <- poverty %>%
  mutate(
    poverty_status = case_when(
      income_less20k == 1 ~ "Below Poverty Line",
      income_less20k == 0 ~ "Above Poverty Line"
    ),
    # Create savings categories based on account amounts
    savings_level = case_when(
      is.na(accts_amt) | accts_amt == 0 ~ "No Savings",
      accts_amt > 0 & accts_amt <= 1000 ~ "Low Savings ($1-$1,000)",
      accts_amt > 1000 & accts_amt <= 10000 ~ "Medium Savings ($1,001-$10,000)",
      accts_amt > 10000 ~ "High Savings ($10,000+)"
    ),
    # Reorder factor levels to: No, Low, Medium, High
    savings_level = factor(savings_level,
                           levels = c("No Savings",
                                      "Low Savings ($1-$1,000)",
                                      "Medium Savings ($1,001-$10,000)",
                                      "High Savings ($10,000+)"))
  ) %>%
  filter(!is.na(income_less20k)) %>%
  group_by(poverty_status, savings_level) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(poverty_status) %>%
```

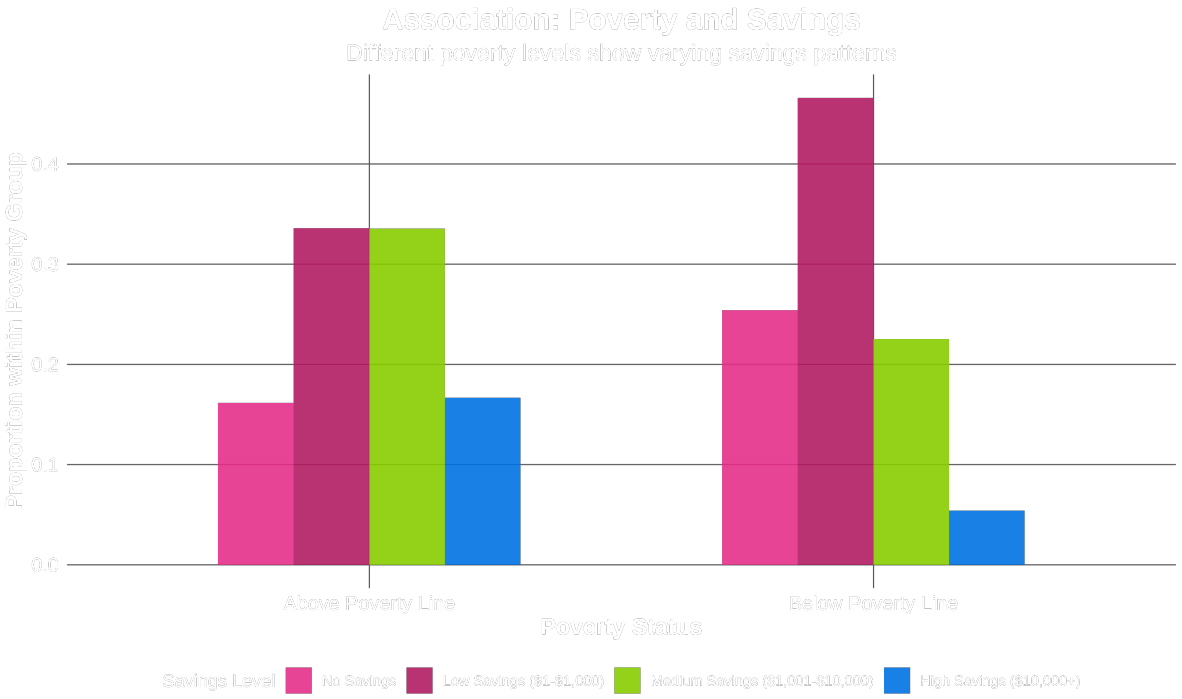
```
  mutate(proportion = count / sum(count)) %>%
  arrange(poverty_status, savings_level)

poverty_savings
```

```
# A tibble: 8 x 4
# Groups:   poverty_status [2]
  poverty_status     savings_level                   count proportion
  <chr>              <fct>                           <int>      <dbl>
1 Above Poverty Line No Savings                      253     0.162
2 Above Poverty Line Low Savings ($1-$1,000)         526     0.336
3 Above Poverty Line Medium Savings ($1,001-$10,000) 525     0.335
4 Above Poverty Line High Savings ($10,000+)         261     0.167
5 Below Poverty Line No Savings                      281     0.254
6 Below Poverty Line Low Savings ($1-$1,000)         515     0.466
7 Below Poverty Line Medium Savings ($1,001-$10,000) 249     0.225
8 Below Poverty Line High Savings ($10,000+)          60     0.0543
```

Visualizing Association: Poverty and Savings

```
Warning: The `size` argument of `element_line()` is deprecated as of ggplot2 3.4.0.
i Please use the `linewidth` argument instead.
```



- Different poverty groups show different savings distributions, suggesting poverty and savings are strongly associated

Testing Associations with Data: Example 2 - Poverty and Cognitive Performance

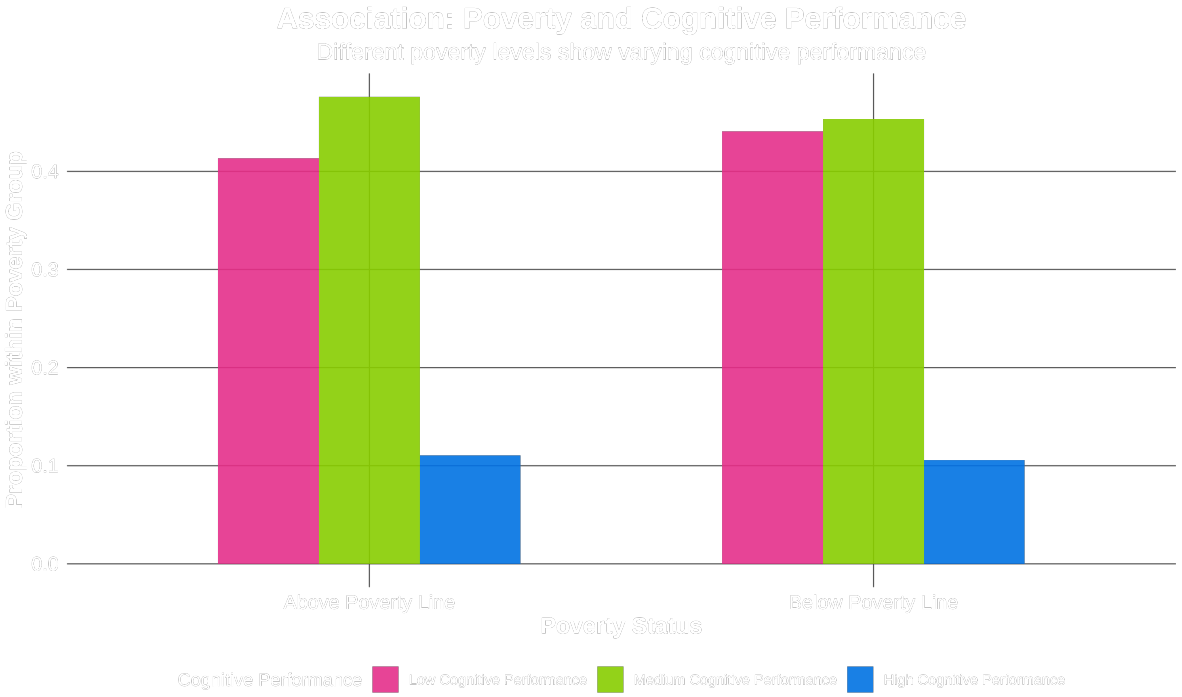
```
# Test association: Does cognitive performance vary by poverty status?
poverty_cognitive <- poverty %>%
  mutate(
    poverty_status = case_when(
      income_less20k == 1 ~ "Below Poverty Line",
      income_less20k == 0 ~ "Above Poverty Line"
    ),
    # Create education proxy based on cognitive performance (stroop test times)
    cognitive_level = case_when(
      stroop_time <= 7.3 ~ "High Cognitive Performance",
      stroop_time > 7.3 & stroop_time <= 7.6 ~ "Medium Cognitive Performance",
      stroop_time > 7.6 ~ "Low Cognitive Performance"
    ),
    # Reorder factor levels to: Low, Medium, High
    cognitive_level = factor(cognitive_level,
```

```
                            levels = c("Low Cognitive Performance",
                                       "Medium Cognitive Performance",
                                       "High Cognitive Performance"))
  ) %>%
  filter(!is.na(stroop_time), !is.na(income_less20k)) %>%
  group_by(poverty_status, cognitive_level) %>%
  summarise(count = n(), .groups = "drop") %>%
  group_by(poverty_status) %>%
  mutate(proportion = count / sum(count)) %>%
  arrange(poverty_status, cognitive_level)

poverty_cognitive
```

```
# A tibble: 6 x 4
# Groups:   poverty_status [2]
  poverty_status     cognitive_level              count proportion
  <chr>              <fct>                        <int>      <dbl>
1 Above Poverty Line Low Cognitive Performance      647      0.413
2 Above Poverty Line Medium Cognitive Performance   745      0.476
3 Above Poverty Line High Cognitive Performance     173      0.111
4 Below Poverty Line Low Cognitive Performance      487      0.441
5 Below Poverty Line Medium Cognitive Performance   501      0.453
6 Below Poverty Line High Cognitive Performance     117      0.106
```

Visualizing Association: Poverty and Cognitive Performance



Weak (no?) association: Less clear relationship between poverty and cognitive performance compared to poverty and savings

Requirement 2: Temporal Ordering

Cause Must Come Before Effect

Seems obvious, but it's often unclear in real data

Examples where timing matters:

Poverty & Crime

[Diagram: Poverty Crime - see HTML slides]

Knowledge & Voting

[Diagram: Knowledge Voting - see HTML slides]

Spending & Votes

[Diagram: Spending Victory - see HTML slides]

The Challenge of Simultaneous Measurement

Cross-sectional data measures every variable at one point in time — so the data alone can't tell us which came first.

```
poverty %>%
  select(treatment, income_less20k, accts_amt, stroop_time, cash) %>%
  head(2)
```

```
# A tibble: 2 x 5
  treatment     income_less20k accts_amt stroop_time  cash
  <chr>                  <dbl>     <dbl>       <dbl> <dbl>
1 Before Payday              0      3000        7.61    30
2 Before Payday              1       800        7.27    75
```

Problem: poverty, savings, cognitive performance, and cash are all measured at once — we can't order them:

- Did poverty lower savings, or did low savings cause poverty?
- Did poverty impair cognitive performance, or the reverse?
- Did more cash improve *both* savings and cognitive performance?

Solutions for Temporal Ordering

Longitudinal Data: Follow the same people over time

Natural Timing: Use events with clear before/after structure

Historical Records: Trace sequences of events (often not credible)

Example: To study if negative ads cause lower campaign donations:

- **Weak**: Survey voters after election about ads and donations
- **Strong**: Measure donations before/after negative ad campaigns begin

Requirement 3: No Confounding

The Biggest Challenge

Confounding occurs when: A third variable Z causes both X and Y, creating a spurious association

Confounding examples:

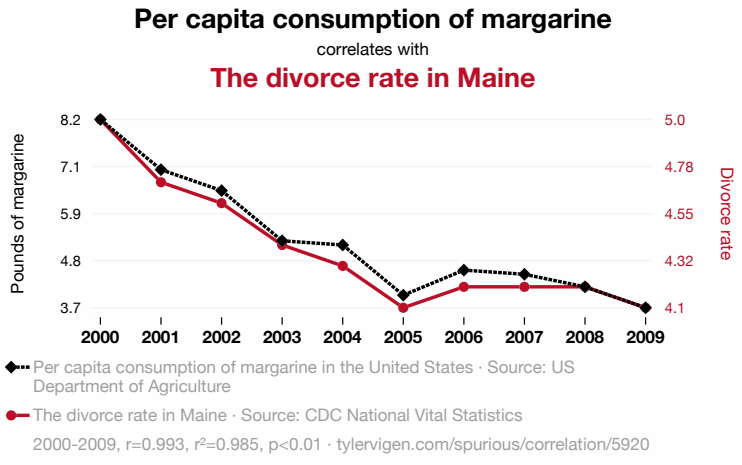
- Campaign Spending → Vote Share (confounder: Candidate Quality)
- Watching 60 Minutes → Death (confounder: Old Age)

[See HTML slides for diagrams]

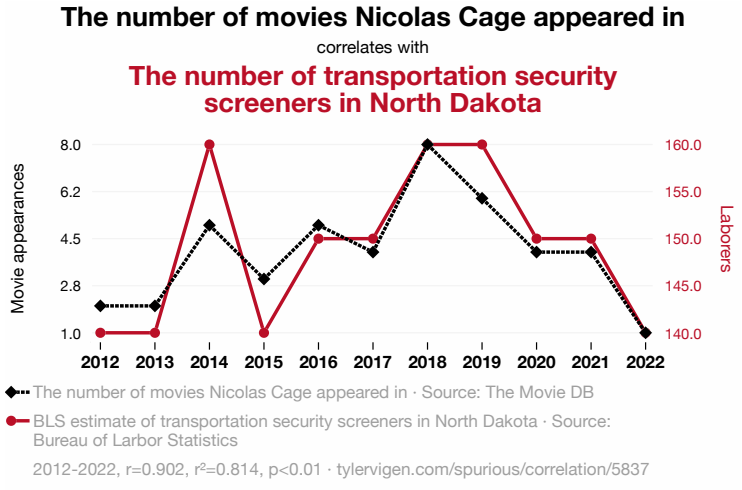
Spurious Correlations

A few more real examples from Tyler Vigen's *Spurious Correlations* project — variables that trend together over time without any plausible causal link between them.

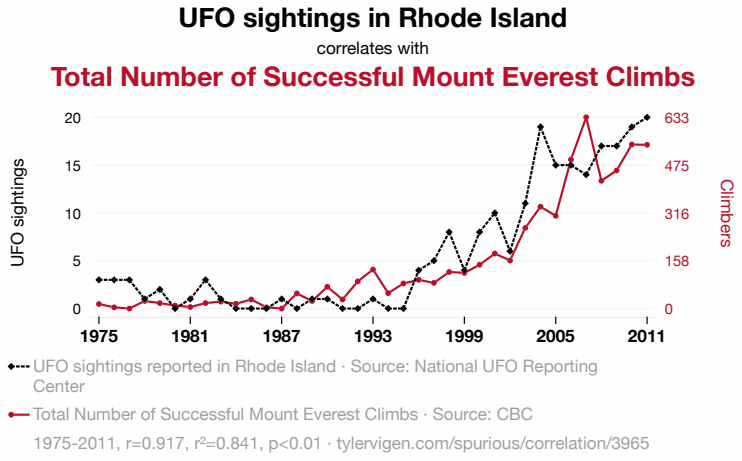
Spurious Correlation: Divorce and Margarine



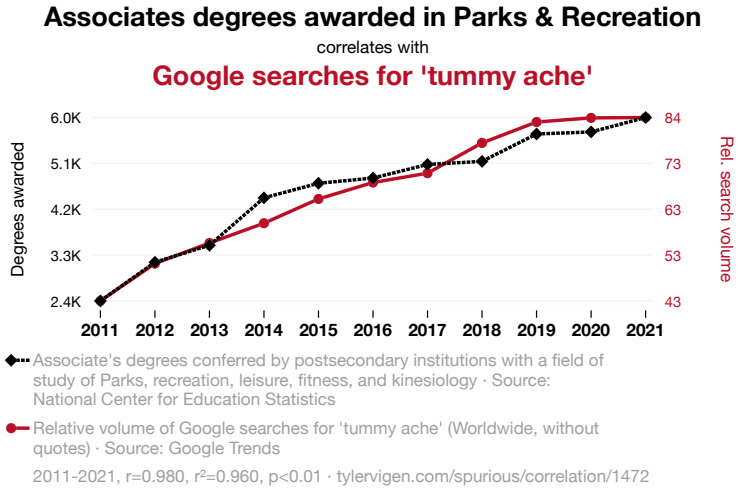
Spurious Correlation: TSA and Nicolas Cage



Spurious Correlation: UFO Sightings and Everest Climbs

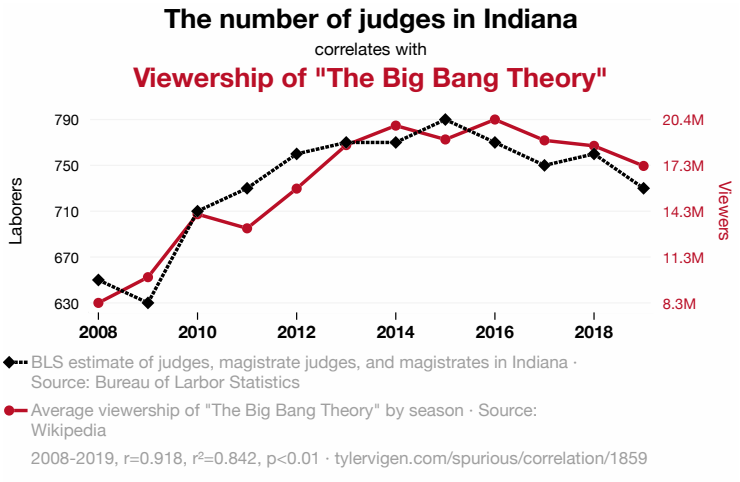


Spurious Correlation: Kinesiology Degrees and "Tummy Ache" Searches

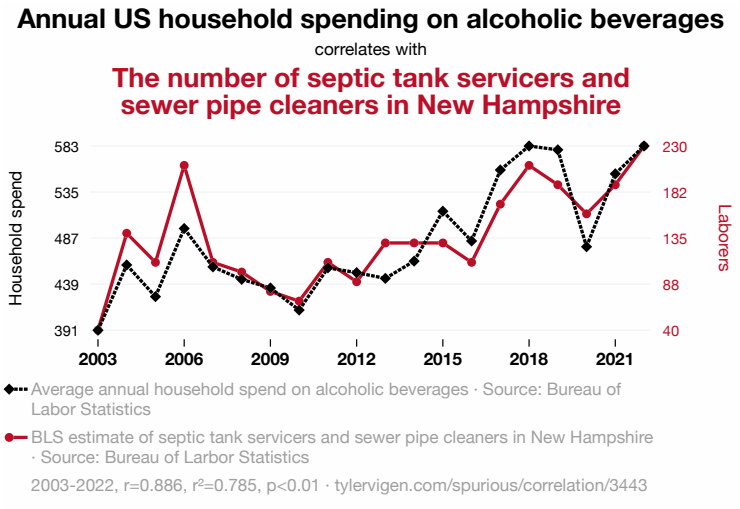


Spurious Correlation: London Gold Prices and Whole Foods Satisfaction

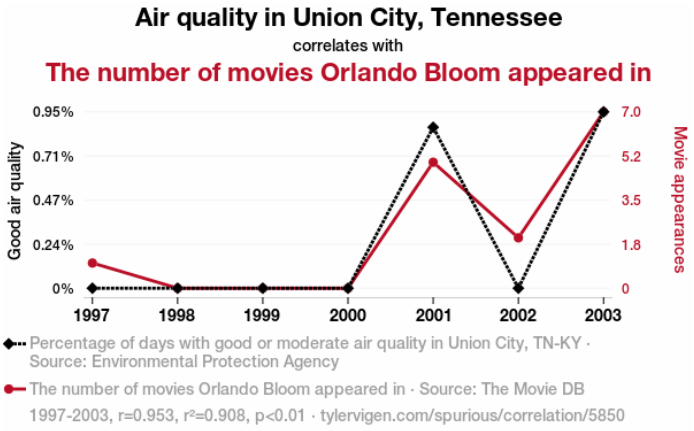
Spurious Correlation: Indiana Judges and *Big Bang Theory* Viewership



Spurious Correlation: Alcohol Spending and NH Septic-Tank Cleaners



Spurious Correlation: Air Quality and Orlando Bloom

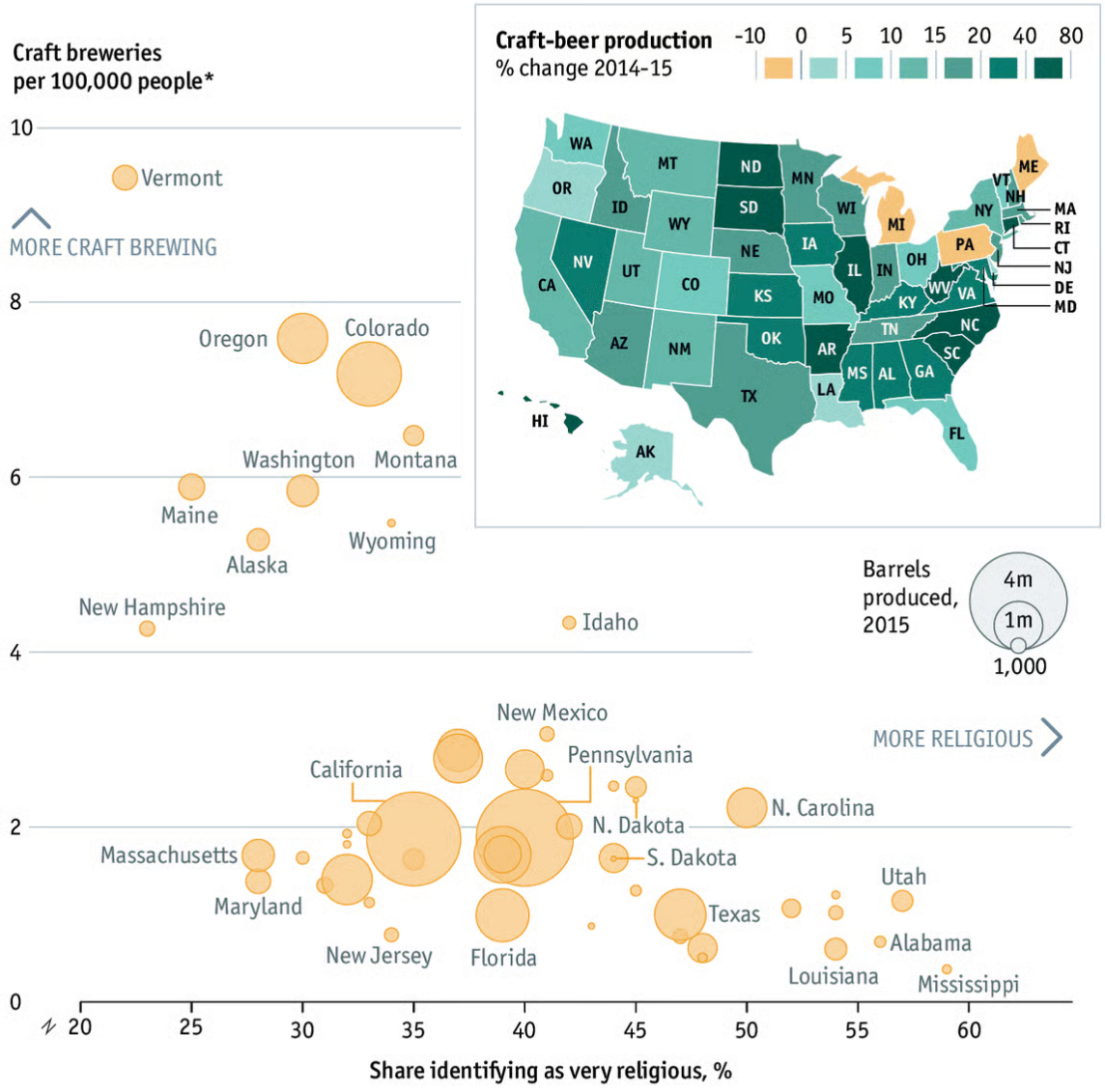


Lesson: Always ask "What else could explain this relationship?"

Craft Beer and Religiosity

Thou shalt not microbrew

United States craft-beer production and religiosity, 2015



Source: Brewers Association; Pew Research Centre

*Aged 21 and over

Economist.com

Identifying Confounders

Common confounders in political science:

- **Socioeconomic status**: Affects voting, education, health, political views
- **Geographic region**: Affects culture, economics, political preferences

- **Age/generation**: Affects technology use, political attitudes, life experiences
- **Media environment**: Affects information, political knowledge, opinions

Always ask: "What else could explain both my cause and my effect?"

Modern Applications: Campaign Finance and Vote Share

A Contemporary Causal Question

Question: "Does campaign spending cause candidates to win more votes?"

The Association: Clear relationship between spending and vote share

```
# Load real campaign finance data
campaign_data <- read_csv("../../data/campaign.csv") %>%
  # Clean and prepare the data
  filter(State=="NH") %>%
  # Convert to more interpretable units
  mutate(
    spending_thousands = total.raised.candidate / 1000,
    vote_share = total.votes ,
    incumbent = ifelse(is.na(prev.elect), 0, 1)  # Create incumbent indicator
  )
```

```
Rows: 2874 Columns: 44
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (8): bonica.rid, election, Cand.ID, Name, State, seat, District, cand.g...
dbl (36): cycle, Party, cfscore, male.winner, running.variable, female.margi...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Show the association
round(cor(campaign_data$spending_thousands, campaign_data$vote_share, use = "complete.obs"),1
```

```
[1] 0.38
```

The correlation coefficient measures the strength and direction of a linear relationship between two variables, ranging from -1 (perfect negative) to $+1$ (perfect positive).

Is This Confounded

```r
# Investigate confounders: Do incumbents spend more AND get more votes?
campaign_data %>%
  group_by(incumbent) %>%
  summarise(
    avg_spending = mean(spending_thousands, na.rm = TRUE),
    avg_vote_share = mean(vote_share, na.rm = TRUE),
    count = n(),
    .groups = "drop"
  ) %>%
  mutate(
    incumbent = ifelse(incumbent == 1, "Incumbent", "Challenger"),
    avg_spending = round(avg_spending, 1),
    avg_vote_share = round(avg_vote_share, 3)
  )
```

```
# A tibble: 2 x 4
  incumbent  avg_spending avg_vote_share count
  <chr>             <dbl>          <dbl> <int>
1 Challenger         53.6           19.2    36
2 Incumbent          87             21.0     9
```

Incumbents spend more AND get more votes

- **Spurious reasoning**: “Spending causes votes”
- **Reality**: “Being incumbent causes both more spending and more votes”

Common Mistakes in Causal Reasoning

Post Hoc, Ergo Propter Hoc

*A **temporal ordering** failure — the cause came first, but order alone isn't proof.*

"After this, therefore because of this"

The Error: Assuming that because B followed A, that A caused B

Examples:

- "I wore my lucky shirt and my team won"
- "The economy improved after the new president took office"
- "Crime dropped after we hired more police"

Why it fails: Presidents inherit economies already on a trajectory, and many things change at once (Fed policy, the business cycle, global trade). Temporal order alone can't isolate the new president's effect.

Remember: Temporal ordering is necessary but not sufficient for causation

Post Hoc in Data: "Our Policy Cut Crime"

A city adopts a "tough on crime" policy in **2019** and credits it for falling crime. But watch the *year-over-year change*:

```
crime <- tibble(
  year       = 2014:2023,
  crime_rate = c(640, 611, 582, 553, 524, 495, 466, 437, 408, 379)
) %>%
  mutate(
    era           = if_else(year >= 2019, "After policy", "Before policy"),
    annual_change = crime_rate - lag(crime_rate)   # change from the prior year
  )

crime %>%
  group_by(era) %>%
  summarise(avg_rate = mean(crime_rate),
            avg_annual_drop = mean(annual_change, na.rm = TRUE))
```

```
# A tibble: 2 x 3
  era           avg_rate avg_annual_drop
  <chr>            <dbl>           <dbl>
1 After policy       437             -29
2 Before policy      582             -29
```

Crime is lower *after* the policy (avg 437 vs 582) — but it fell by the **same 29 points every year**, before *and* after 2019. The trend never changed; the policy added nothing. *(Illustrative data.)*

Reverse Causation

*Another **temporal ordering** trap — the effect may actually come first.*

Getting the direction of causation backwards

Examples:

- Does happiness cause success, or success cause happiness?
- Do good policies cause economic growth, or economic growth enable good policies?

Why it fails: A cross-sectional correlation between growth and "good policies" can't tell us which came first — wealthy countries can afford better institutions, and better institutions also help countries grow wealthy. Without temporal data we can't separate cause from consequence.

Reverse Causation in Data: Police and Crime

Cities with more police have more crime — do police *cause* crime? The record shows last year's crime, officers added this year, and this year's crime:

```
cities <- tibble(
  city            = c("Akron", "Belmont", "Carver", "Dover", "Elgin",
                      "Fenton", "Girard", "Hollis", "Ipswich", "Joplin"),
  crime_last_year = c(28, 44, 33, 61, 22, 55, 39, 66, 25, 50),  # per 10k, year t-1
  police_added    = c( 8, 16, 10, 20,  7, 17, 14, 23,  9, 18),  # officers added, year t
  crime_this_year = c(31, 45, 30, 58, 25, 57, 41, 63, 24, 52)   # per 10k, year t
)

# Naive: more police go with more crime
round(cor(cities$police_added, cities$crime_this_year), 2)
```

```
[1] 0.97
```

```
# But hiring RESPONDS to last year's crime
round(cor(cities$crime_last_year, cities$police_added), 2)
```

```
[1] 0.98
```

Both correlations are strongly positive — the police–crime link is real but **reversed**: departments add officers *because* crime was high last year. Crime came first. *(Illustrative data.)*

Selection Bias Disguised as Causation

*A cousin of **confounding** — the groups differ for reasons other than the supposed cause.*

The Problem: Comparing groups that chose to be different

Examples:

- "Private school students have higher test scores" (ignoring family background)
- "People who exercise live longer" (ignoring health consciousness)
- "Countries with democracy are more prosperous" (ignoring development level)

Why it fails: Families who pay for private school differ from public-school families in income, parental education, and engagement — all of which independently raise test scores. The "school effect" is confounded with everything that led families to choose private school.

Selection Bias in Data: Occupation and Suicide Terrorism

Robert Pape studied **only suicide-terror campaigns** and found nearly all occurred under a **foreign military occupation** — concluding occupation drives terrorism. The catch: he sampled on the *outcome*.

```
# Foreign occupations AND comparable non-occupations
conflicts <- tibble(
  foreign_occupation = c(rep("Occupied", 12), rep("Not occupied", 8)),
  suicide_campaign   = c(rep("Yes", 5), rep("No", 7), rep("Yes", 1), rep("No", 7))
)

# The full 2x2: occupation status by whether a campaign occurred
conflicts %>% count(foreign_occupation, suicide_campaign)
```

```
# A tibble: 4 x 3
  foreign_occupation suicide_campaign     n
  <chr>              <chr>            <int>
1 Not occupied       No                   7
2 Not occupied       Yes                  1
3 Occupied           No                   7
4 Occupied           Yes                  5
```

Pape saw only the `Yes` rows: 5 of 6 attacks were under occupation — "occupation causes terror." But the full table shows most occupations (7 of 12) had **no** campaign. **Selecting on the dependent variable** hides the `No` rows and manufactures the link. *(Illustrative data.)*

Ecological Fallacy: Groups vs. Individuals

A relationship measured on **groups** (counties, states) can point the *opposite* way from the same relationship among **individuals**.

```
# Democratic vote share by income, at two levels of analysis
vote_by_income <- tibble(
  income_level   = c("Low", "Middle", "High"),
  indiv_pct_dem  = c(61, 50, 39),   # among individual voters
  county_pct_dem = c(43, 50, 58)    # among counties at that income
)

vote_by_income
```

```
# A tibble: 3 x 3
  income_level indiv_pct_dem county_pct_dem
  <chr>                <dbl>          <dbl>
1 Low                     61             43
2 Middle                  50             50
3 High                    39             58
```

Same variables, opposite slopes: rich *voters* trend Republican (61→39), but rich *counties* — urban, coastal — trend Democratic (43→58). Reading individuals off county totals is the **ecological fallacy**. *(Illustrative of the real U.S. income–vote paradox.)*

Necessary vs. Sufficient Causes

Most political causes are neither necessary nor sufficient — they only *raise the odds*.

- **Necessary**: Y can't happen *without* X (no oxygen → no fire)
- **Sufficient**: X *alone guarantees* Y (a guillotine is sufficient for death)

	Sufficient	**Not sufficient**
Necessary	Winning number → lottery win	Oxygen → fire
Not necessary	Guillotine → death	Campaign spending → victory

Most causes in politics are **contributory** — they raise the probability without being required or decisive. That's why one counterexample ("my grandfather smoked and lived to 95") does *not* refute a causal claim.

Using AI to Identify Alternative Explanations

AI as Your Confounding Detective

Effective prompt for identifying confounders:

> "I found that cities with more coffee shops have higher voter turnout. Before concluding that coffee shops increase civic engagement, help me brainstorm what other variables might cause both coffee shop density AND voter turnout. Think about demographics, economics, and culture."

AI for Causal Critique

Research design critique prompt:

> "Evaluate this causal claim: 'Social media use reduces political knowledge because people who spend more time on social media score lower on political knowledge tests.' What are the three requirements for causality, and does this study meet them? What alternative explanations should I consider?"

Best Practices for Causal Claims

Red Flags in Causal Arguments

Be skeptical when you see:

- "Studies show X causes Y" (without mentioning study design)
- Causal claims from cross-sectional data
- No discussion of alternative explanations
- Dramatic policy recommendations from single studies

Strengthening Causal Arguments

Use multiple lines of evidence:

- Replicate findings across different populations
- Test with different research designs
- Look for natural experiments
- Check for temporal consistency

Acknowledge limitations:

- Be explicit about what you can and cannot conclude
- Discuss alternative explanations
- Admit when evidence is merely suggestive

Key Concepts to Remember

- **Correlation Causation** - but association is the first requirement
- **Three requirements**: association, temporal ordering, no confounding
- **Confounding is everywhere** - always ask "what else could explain this?"
- **Watch the classic traps**: post hoc, reverse causation, and selection bias
- **AI can help** - but only if you ask the right questions

Looking Ahead

In Our Next Class

Modern Causal Inference & Linear Regression

- The fundamental problem of causal inference and potential outcomes
- Randomized experiments and difference-in-differences
- Fitting and interpreting your first regression line